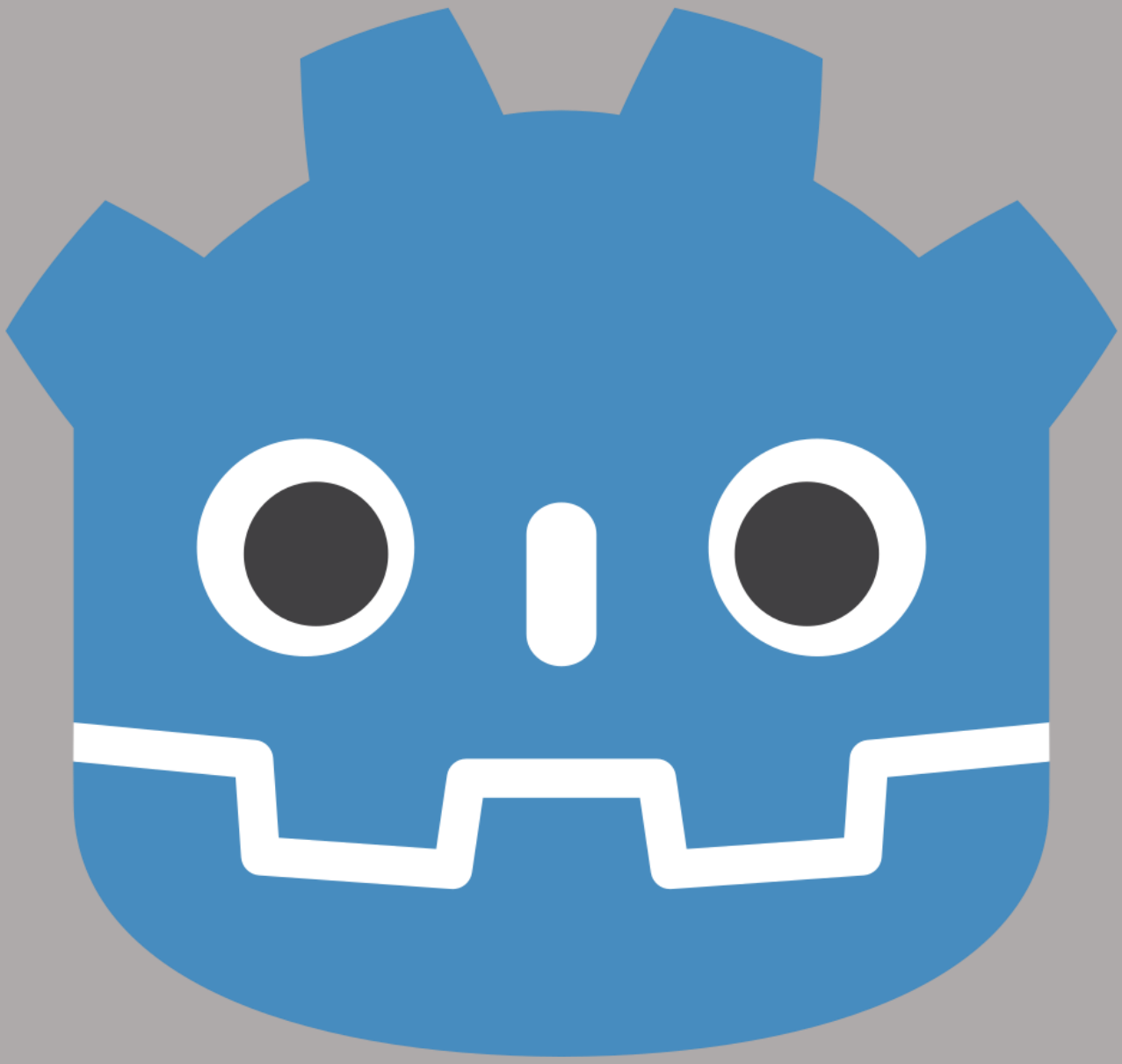
GODOT ENGINE DOCUMENTATION



Actualized for version 4.6 Stable

CONTENT

Godot Docs Content

1 2D Topics

1.1 2D Animation Topic

1.1.X AnimatedSprite2D (+)

1.1.X Skeleton2D Animation (+)

1.2 2D Physics Topic

1.2.X RigidBody2D (+)

1.2.X StaticBody2D (+)

1.2.X CharacterBody2D (+)

1.2.X Area2D (+)

1.2.X CollisionShape2D (+)

1.2.X CollisionPolygon2D (+)

1.2.X PinJoint2D (+)

1.2.X GrooveJoint2D (+)

1.2.X DampedSpringJoint2D [works really weird] (+)

1.2.X Godot Rapier2D Physics (+)

2 3D Topics

2.1 3D Animation Topic

2.2 3D Physics Topic

3 Audio

4 Exporting/Building Projects

Godot Engine 4.6 (stable) provides different platforms to build/export project: Android, iOS, Linux, MacOS, Web-Platform, Windows PC, etc.

In order to build/export project, user should go to Project/Export… menu, it will show export window. Next step is to add any platform you need to build project with; you just need to add any of available presets. At the first time in building/exporting project it will probably show some errors, like: **No export template found at the expected path**. You just simply need to go to **Manage Export Template** section: Editor/Manage Export Templates… And download export templates.

Resource options

When exporting, Godot makes a list of all the files to export and then creates the package. There are **5 different modes** for exporting:

- Export all resources in the project
- Export selected scenes (and dependencies)
- Export selected resources (and dependencies)
- Export all resources in the project except resources checked below
- Export as dedicated server

Export all resources in the project will export every resource in the project. **Export selected scenes** and **Export selected resources** gives you a list of the scenes or resources in the project, and you have to select every scene or resource you want to export.

Export as dedicated server will remove all visuals from a project and replace them with a placeholder. This includes Cubemap, CubemapArray, Material, Mesh, Texture2D, Texture2DArray, Texture3D. You can also go into the list of files and specify specific visual resources that you do wish to keep.

Configuration files

The export configuration is stored in two files that can both be found in the project directory:

- **export_presets.cfg**: This file contains the vast majority of the export configuration and can be safely committed to version control. There is nothing in here that you would normally have to keep secret.
- **.godot/export_credentials.cfg**: This file contains export options that are considered confidential, like passwords and encryption keys. It should generally **not** be committed to version control or shared with others unless you know exactly what you are doing.

Since the credentials file is usually kept out of version control systems, some export options will be missing if you clone the project to a new machine. The easiest way to deal with this is to copy the file manually from the old location to the new one. [1]

4.1 Exporting to Android

4.1.X Android Export Useful Links

1. Godot wireless debugging android: https://youtu.be/jebwA5eB-sE?si=Qi8OSQb9qNK3-M3T
2. Remote Debug for Android topic: https://forum.godotengine.org/t/remote-debug-for-andriod/40418/5

5 File and Data I/O

6 Import Pipeline

7 Internationalization

8 Math

9 Networking

10 Performance

11 Platform-specific

12 Plugins

13 Godot GDScript

GDScript is a high-level, object-oriented, imperative, and gradually typed programming language built for Godot. It uses an indentation-based syntax similar to languages like Python. Its goal is to be optimized for and tightly integrated with Godot Engine, allowing great flexibility for content creation and integration.

GDScript is entirely independent from Python and is not based on it.

13.1 Beginner Topic

13.2 Intermediate Topic

14 C# Scripting

C# is a high-level programming language developed by Microsoft. Godot supports C# as an option for a scripting language, alongside Godot's own GDScript. [1]

C# is a modern, innovative, open-source, cross-platform object-oriented programming language and one of the top 5 programming languages on GitHub. [2]

14.1 Beginner Topic

Beginner-friendly topic for developers, who recently started to work with C#/.NET.

This topic includes such sections:

- Variables and Constants
- Data Types
- Operations and Expressions
- Statements
- Loops
- Arrays
- Collections
- Functions/Methods
- Switch Construction
- Enums
- Object oriented programming sections

14.1.1 Variables and Contants

Variables are used to store data in a program. A variable represents a named memory location that stores a value of a specific type. A variable has a type, a name, and a value. The type determines what kind of information the variable can store.

A variable must be defined before use. The syntax for defining a variable is as follows:

```
1. type variableName;
```

First comes the variable type, then its name. A variable name can be any arbitrary name that meets the following requirements:

The name can contain any numbers, letters, and underscores, but the first character in the name must be a letter or underscore.

The name cannot contain punctuation or spaces.

The name cannot be a C# keyword. Such words are rare, and when working in Visual Studio, the development environment highlights keywords in blue.

Although variable names can be anything, they should be given descriptive names that convey their purpose.

For example, let's define a simple variable:

```
1. string name;
```

In this case, the variable name is defined as a string. This variable represents a string. Since the variable definition is a statement, it is followed by a semicolon.

It's important to note that C# is a case-sensitive language, so the following two variable definitions will represent two different variables:

```
1. string name;
2. string Name;
```

After defining a variable, we can assign a value:

Since the variable name represents a string type, we can assign it a string in double quotes. Furthermore, a variable can only be assigned a value that matches its type.

Further, we can use the variable name to access the memory location where its value is stored.

We can also assign a value to a variable immediately upon definition. This technique is called initialization:

```
1. string name = “Tom”;
```

A distinctive feature of variables is that their value can be changed multiple times within a program. For example, let's create a small program in which we define a variable, change its value, and print it to the console:

```
1. public override void _Ready()
2. {
3.     string name = "Tom";
4.
5.     GD.Print(name);
6.
7.     name = "Bob";
8.     GD.Print(name);
9. }
```

The console output is:

```
Godot Engine v4.6.stable.mono.official.89cea1439 - https://godotengine.org
Vulkan 1.4.325 - Forward Mobile - Using Device #0: NVIDIA - NVIDIA GeForce RTX 4060 Ti

Tom
Bob
```

Constants

A distinctive feature of variables is that their value can be changed while the program is running. However, C# also has constants. A constant must be initialized when defined, and once defined, its value cannot be changed.

Constants are used to describe values, that should not change in a program. Constants are defined using the const keyword, which precedes the constant type:

```
1. const string NAME = "Tom";
```

So, in this case, the constant NAME is defined, which stores the string "Tom." Constant names are often uppercase, but this is merely a convention.

When using constants, remember that we can only declare them once and that they must be defined by compile time. For example, in the following case, we'll get an error because the constant hasn't been assigned an initial value:

```
1. const string NAME; // ! Error - constant NAME is not initialized
```

Moreover, we won’t be able to change it while the program is running:

```
1. const string NAME = "Tom"; // define a constant
2. NAME = "Bob"; // !Error - the constant's value cannot be changed
```

Thus, if we need to store data in a program but it shouldn't be changed, it is defined as constants. If it can be changed, it is defined as variables.

14.1.2 Data Types

Like many programming languages, C# has its own data type system, which is used to create variables. A data type defines the internal representation of data, the range of values an object can accept, and the valid actions that can be performed on the object.

C# has the following basic data types:

- **bool:** stores the value true or false (logical literals). Represented by the System.Boolean system type;

```
1. bool alive = true;
2. bool isDead = false;
```

- **byte:** stores an integer from 0 to 255 and occupies 1 byte. Represented by the System.Byte system type;

```
1. byte bit1 = 1;
2. byte bit2 = 102;
```

- **sbyte:** stores an integer between -128 and 127 and occupies 1 byte. Represented by the System.SByte system type;

```
1. sbyte bit1 = -101;
2. sbyte bit2 = 102;
```

- **short:** stores an integer from -32768 to 32767 and occupies 2 bytes. Represented by the System.Int16 system type;

```
1. short n1 = 1;
2. short n2 = 102;
```

- **ushort:** stores an integer from 0 to 65535 and occupies 2 bytes. Represented by the System.UInt16 system type;

```
1. ushort n1 = 1;
2. ushort n2 = 102;
```

- **int:** stores an integer from -2147483648 to 2147483647 and occupies 4 bytes. Represented by the System.Int32 system type. All integer literals by default represent values of the int type:

```
1. int a = 10;
2. int b = 0b101; // binary form b = 5
3. int c = 0xFF; // hexadecimal form c = 255
```

- **uint:** stores an integer from 0 to 4294967295 and occupies 4 bytes. Represented by the System.UInt32 system type.

```
1. uint a = 10;
2. uint b = 0b101;
3. uint c = 0xFF;
```

- **long:** stores an integer from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 and occupies 8 bytes. Represented by the System.Int64 system type.

```
long a = -10;
long b = 0b101;
long c = 0xFF;
```

- **ulong:** stores an integer from 0 to 18,446,744,073,709,551,615 and occupies 8 bytes. Represented by the System.UInt64 system type.

```
ulong a = 10;
ulong b = 0b101;
ulong c = 0xFF;
```

- **float:** stores a floating-point number from -3.4*1038 to 3.4*1038 and occupies 4 bytes. Represented by the System.Single system type.
- **double:** stores a floating-point number from ±5.0*10-324 to ±1.7*10308 and occupies 8 bytes. Represented by the System.Double system type.
- **decimal:** stores a decimal fractional number. When used without a decimal point, it has a value from ±1.0*10-28 to ±7.9228*1028, can store 28 decimal places, and occupies 16 bytes. Represented by the System.Decimal system type.
- **char:** stores a single Unicode character and occupies 2 bytes. Represented by the System.Char system type. Character literals correspond to this type:

```
char a = 'A';
char b = '\x5A';
char c = '\u0420';
```

- **string:** stores a set of Unicode characters. Represented by the System.String system type. This type corresponds to string literals.

```
string hello = "Hello";
string word = "world";
```

- **object:** Can store a value of any data type and occupies 4 bytes on a 32-bit platform and 8 bytes on a 64-bit platform. It is represented by the System.Object system type, which is the base type for all other .NET types and classes.

```
object a = 22;
object b = 3.14;
object c = "hello code";
```

For example, let's define several variables of different types and print their values to the console:

```
string name = "Tom";
int age = 33;
bool isEmployed = false;
double weight = 78.65;

GD.Print($"Name: {name}");
GD.Print($"Age: {age}");
GD.Print($"Weight: {weight}");
GD.Print($"Is employed: {isEmployed}");
```

To output data to the console, interpolation is used here: the $ sign is placed before the string, and then we can enter variable values in the string within curly brackets. The program's console output:

```
Godot Engine v4.6.1.stable.mono.official.14d19694e - https://godotengine.org
Vulkan 1.4.325 - Forward Mobile - Using Device #0: NVIDIA - NVIDIA GeForce RTX 4060 Ti

Name: Tom
Age: 33
Weight: 78,65
Is employed: False
```

Using Suffixes

When assigning values, keep the following nuance in mind: all real literals (fractional numbers) are treated as **double** values. To indicate that a fractional number represents a float or decimal, a suffix must be added to the literal: F/f for **float** and M/m for **decimal**.

```
float a = 3.14F;
float b = 30.6f;

decimal c = 1005.8M;
decimal d = 334.8m;
```

Similarly, all integer literals are treated as values of type **int**. To explicitly indicate that an integer literal represents a uint value, use the suffix **U/u**, for the **long** type, the suffix **L/l**, and for the **ulong** type, the suffix **UL/ul**:

```
uint a = 10U;
long b = 20L;
ulong c = 30UL;
```

Using System Types

Above, when listing all the basic data types, each one was referred to by its system type. This is because the name of a built-in type is essentially a shorthand for a system type. For example, the following variables would be equivalent in type:

```
int a = 4;
System.Int32 b = 4;
```

Implicit Typing

Previously, we explicitly specified the variable type, for example, int x; And the compiler already knew at runtime that x stores an integer value.

However, we can also use the implicit typing model:

```
1. var hello = "Hell to World";
2. var c = 20;
```

For implicit typing, the var keyword is used instead of the data type name. Then, during compilation, the compiler infers the data type based on the assigned value. Since all integer values are treated as int by default, the variable c will be of type int. Similarly, if the variable hello is assigned a string, this variable will be of type string.

These variables are similar to regular variables, but they have some limitations.

First, we cannot declare an implicitly typed variable and then initialize it:

```
1. // this code works
2. int a;
3. a = 20;
4.
5. // this code does not work
6. var c;
7. c = 20;
```

Secondly, we cannot specify null as the value of an implicitly typed variable:

```
1. // this code does not work
2. var c = null;
```

Since the value is null, the compiler will not be able to infer the data type.

14.1.3 Operations and Expressions

Arithmetic operations in the C# language

C# uses most of the same operators as other programming languages. Operations represent specific actions performed on operands—the participants in the operation. An operand can be a variable or a value (such as a number). Operations can be unary (performed on one operand), binary (performed on two operands), or ternary (performed on three operands). Let's look at all types of operations.

Binary arithmetic operations:

- **+**

  Addition of two numbers:

```
1. int x = 10;
2. int z = x + 12; // 22
```

- **-**

  Subtraction of two numbers:

```
1. int x = 10;
2. int z = x - 6; // 4
```

- *****

  Multiplication of two numbers:

```
1. int x = 10;
2. int z = x * 5; // 50
```

- **/**

  Division operation for two numbers:

```
1. int x = 10;
2. int z = x / 5; // 2
3.
4. double a = 10;
5. double b = 3;
6. double c = a / b; // 3.33333333
```

When dividing, it is worth considering that if both operands represent integers, the result will also be rounded to an integer:

```
1. double z = 10 / 4; // result is 2
```

Although the result of the operation is ultimately placed in a double variable, which allows the fractional part to be preserved, the operation itself involves two literals, which by default are treated as int objects (i.e., integers), and the result will also be an integer.

To resolve this situation, it is necessary to define the literals or variables involved in the operation as double or float:

```
1. double z = 10.0 / 4.0; //result is 2.5
```

- **%**

  Operation to obtain the remainder of an integer division of two numbers:

```
1. double x = 10.0;
2. double z = x % 4.0; //result is 2
```

There are also a number of unary operations that involve one operand:

- **++**

  Increment Operator

  There are prefix increments: ++x - first, the value of variable x is incremented by 1, and then its value is returned as the result of the operation.

  There is also a postfix increment: x++ - first, the value of variable x is returned as the result of the operation, and then 1 is added to it.

```
1. int x1 = 5;
2. int z1 = ++x1; // z1=6; x1=6
3. GD.Print($"{x1} - {z1}");
4.
5. int x2 = 5;
6. int z2 = x2++; // z2=5; x2=6
7. GD.Print($"{x2} - {z2}");
```

- **--**

  The decrement operator, or decrease, a value by one. There is also a prefix decrement (--x) and a postfix decrement (x--).

```
1. int x1 = 5;
2. int z1 = --x1; // z1=4; x1=4
3. GD.Print($"{x1} - {z1}");
4.
5. int x2 = 5;
6. int z2 = x2--; // z2=5; x2=4
7. GD.Print($"{x2} - {z2}");
```

When performing multiple arithmetic operations simultaneously, consider the order in which they are performed. The order of precedence from highest to lowest is:

1. Increment, decrement
2. Multiplication, division, remainder
3. Addition, subtraction

Parentheses are used to change the order of operations.

Let's look at a set of operations:

```
1. int a = 3;
2. int b = 5;
3. int c = 40;
4. int d = c---b*a; // a=3 b=5 c=39 d=25
5. GD.Print($"a={a} b={b} c={c} d={d}");
```

Here we're dealing with three operations: decrement, subtraction, and multiplication. First, the variable c is decremented, then b*a is multiplied, and finally, the subtraction. So, the actual set of operations looks like this:

```
1. int d = (c--)-(b*a);
```

But using parentheses we could change the order of operations, like this:

```
1. int a = 3;
2. int b = 5;
3. int c = 40;
4. int d = (c-(--b))*a; // a=3 b=4 c=40 d=108
5. GD.Print($"a={a} b={b} c={c} d={d}");
```

Operator Associativity

As noted above, multiplication and division operations have the same precedence, but what then is the result in the expression:

```
1. int x = 10 / 5 * 2;
```

Should we interpret this expression as (10 / 5) * 2 or as 10 / (5 * 2)? Depending on how we interpret it, we'll get different results.

When operators have the same precedence, the order of evaluation is determined by the associativity of the operators. Depending on associativity, there are two types of operators:

- Left-associative operators, which are evaluated from left to right
- Right-associative operators, which are evaluated from right to left

All arithmetic operators are left-associative, meaning they are evaluated from left to right. Therefore, the expression 10 / 5 * 2 must be interpreted as (10 / 5) * 2, which results in 4.

Bitwise operations

A special class of operations are bitwise operations. They are performed on individual digits of a number. In this regard, numbers are considered in binary representation; for example, 2 in binary representation is 10 and has two digits, while the number 7 is 111 and has three digits.

Logical operations

- **&**(logical multiplication)
  Multiplication is performed bitwise, and if both operands have bit values equal to 1, the operation returns 1; otherwise, it returns 0. For example:

```
1. int x1 = 2; //010
2. int y1 = 5;//101
3. GD.Print(x1&y1); // output 0
4.
5. int x2 = 4; //100
6. int y2 = 5; //101
7. GD.Print(x2 & y2); // output 4
```

In the first case, we have two numbers, 2 and 5. 2 represents 010 in binary, and 5 represents 101. We multiply the numbers bitwise (0*1, 1*0, 0*1) and end up with 000.

In the second case, instead of 2, we have the number 4, which has a 1 in the first digit, just like 5, so we end up with (1*1, 0*0, 0*1) = 100, which is the number 4 in decimal format.

- **|** (logical addition)
  Similar to logical multiplication, the operation is also performed on binary digits, but now it returns a one if at least one number in a given digit has a one. For example:

```
1. int x1 = 2; //010
2. int y1 = 5;//101
3. GD.Print(x1|y1); // ouput 7 - 111
4. int x2 = 4; //100
5. int y2 = 5;//101
6. GD.Print(x2 | y2); // output 5 - 101
```

- **^** (logical exclusive OR)
  This operation is also called XOR and is often used for simple encryption:

```
1. int x = 45; // Value, that need to encrypt - in binary form 101101
2. int key = 102; //Key - in binary form 1100110
3.
4. int encrypt = x ^ key; // Result will be a number from 1001011 to 75
5. GD.Print($"Encrypted number: {encrypt}");
6.
7. int decrypt = encrypt ^ key; // Result will be the original number 45
8. GD.Print($"Decrypted number: {decrypt}");
```

Here, bitwise operations are performed again. If the current digit values of both numbers are different, then 1 is returned; otherwise, 0 is returned:

```
1. 45 ^ 102 =
2. 0101101
3. ^
4. 1100110
5. =
6. 1001011
7. = 75
```

So, from 45 ^ 102 we get the number 75 as the result. And to decipher the number, we apply the same operation to the result.

Similarly, you can exchange two positive numbers without using an additional variable:

```
int a = 9; // 1001
int b = 5; // 0101

a = a ^ b; // a = 1001 ^ 0101 = 1100 = 12
b = a ^ b; // b = 12 ^ 5 = 1100 ^ 0101 = 1001 = 9
a = a ^ b; // a = 12 ^ 9 = 1100 ^ 1001 = 0101 = 5

GD.Print($"a: {a}") ; // 5
GD.Print($"b: {b}") ; // 9
```

- ~ (logical negation or inversion)
  Another bitwise operation that inverts all digits: if a digit's value is 1, it becomes 0, and vice versa.

```
int x = 12;         // 00001100
GD.Print(~x);       // 11110011   or -13
```

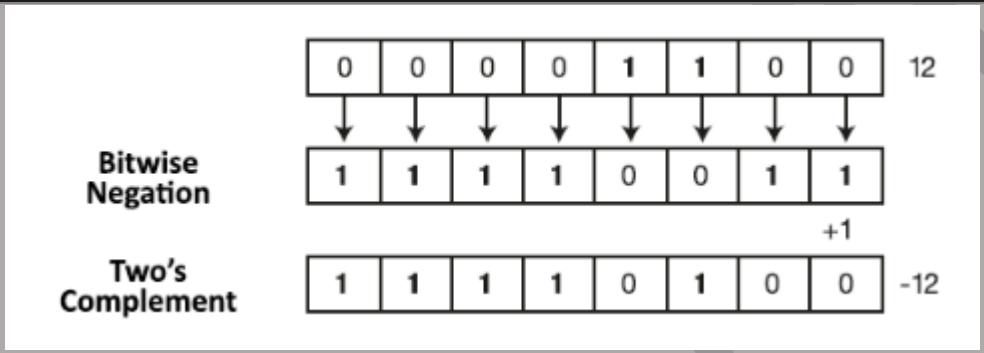
Representing Negative Numbers

Signed numbers in C# are written using **two's complement notation**, where the most significant digit is the sign digit. If its value is 0, the number is positive, and its binary representation is the same as that of an unsigned number. For example, 0000 0001 in decimal is 1.

If the most significant digit is 1, then we're dealing with a negative number. For example, 1111 1111 in decimal represents -1. Accordingly, 1111 0011 represents -13.

To convert a positive number to a negative number, invert it and add one:

```
1. int x = 12;
2. int y = ~x;
3. y += 1;
4. GD.Print(y);   // -12
```



Shift operations

Shift operations are also performed on the digits of numbers. Shifts can be performed to the right or left.

- x << y - shifts the number x left by y digits. For example, 4 << 1 shifts the number 4 (which is 100 in binary) one digit to the left, resulting in 1000, or 8 in decimal.
- x >> y - shifts the number x right by y digits. For example, 16 >> 1 shifts the number 16 (which is 10000 in binary) one digit to the right, resulting in 1000, or 8 in decimal.

Thus, if the original number to be shifted is divisible by two, the resulting shift is effectively a multiplication or division by two. Therefore, this operation can be used instead of direct multiplication or division by two. For example:

```
1. int a = 16; // in binary form 10000
2. int b = 2; // in binary form
3. int c = a << b; // Shift number 10000 left to 2 digits, equals 1000000 or 64 in binary system
4.
5. GD.Print($"Encrypted number: {c}") ;    // 64
6.
7. int d = a >> b; // Shift number 10000 right to 2 digits, equals 100 or 4 in binary system
8. GD.Print($"Encrypted number: {d}");     // 4
```

In this case, the numbers involved in the operations do not necessarily have to be multiples of 2:

```
1. int a = 22; // in binary form 10110
2. int b = 2; // in binary form
3. int c = a << b; // Shift number 10110 left to 2 digits, equals 1011000 or 88 in binary system
4.
5. GD.Print($"Encrypted number: {c}") ;    // 88
6.
7. int d = a >> b; // Shift number 10110 right to 2 digits, equals 101 or 5 in binary form
8. GD.Print($"Encrypted number: {d}");     // 5
```

An example of a practical application of bitwise operations

Many people underestimate bitwise operations and don't understand their purpose. However, they can help solve a number of problems. First of all, they allow us to manipulate data at the level of individual bits. Here's an example: We have three numbers ranging from 0 to 3:

```
int value1 = 3;  // 0b0000_0011
int value2 = 2;  // 0b0000_0010
int value3 = 1;  // 0b0000_0001
```

We know that the values of these numbers will not be greater than 3, and we need to compress this data as much as possible. We can store three numbers into a single number. Bitwise operations will help us with this.

```
int value1 = 3;  // 0b0000_0011
int value2 = 2;  // 0b0000_0010
int value3 = 1;  // 0b0000_0001
int result = 0b0000_0000;
// saving in result values from value1
result = result | value1; // 0b0000_0011
// shifting digits in result to 2 digits left
result = result << 2;   // 0b0000_1100
// saving in result values from value2
result = result | value2;  // 0b0000_1110
// shifting digits in result to 2 digits left
result = result << 2;   // 0b0011_1000
// saving in result values from value3
result = result | value3;  // 0b0011_1001

GD.Print(result);  // 57
```

Let's break down this code. First, we define all the numbers to be stored: value1, value2, and value3. The result variable is defined to store the result, which defaults to 0. For clarity, it's assigned a binary value:

```
int result = 0b0000_0000;
```

Saving first number in result:

```
result = result | value1; // 0b0000_0011
```

Here we are dealing with the logical operation of bitwise addition - if one of the corresponding bits is equal to 1, then the resulting bit will also be equal to 1. That is, in fact,

```
0b0000_0000
+
0b0000_0011
=
0b0000_0011
```

So, the first number is stored in result. We'll store the numbers in order. That is, the first number will appear in result, then the second, and then the third. Therefore, we shift result two places to the left (our numbers take up no more than two places in memory):

```
result = result << 2;   // 0b0000_1100
```

So, in fact:

```
0b0000_0011 << 2 =
0b0000_1100
```

Next, we repeat the logical addition operation, storing the second number:

```
result = result | value2;  // 0b0000_1110
```

That is equivalent to:

```
0b0000_1100
+
0b0000_0010
=
0b0000_1110
```

Next, we repeat the shift to the left by two places and store the third number. The resulting binary representation is 0b0011_1001. In decimal, this number is 57. But this doesn't matter, because we're interested in the specific bits of the number. It's worth noting that we've stored three numbers in a single number, and there's still some space left in the result variable. In reality, the exact number of bits stored isn't important. In this example, we'll only store two bits.

To recover data, we will resort to the reverse order:

```
result = 0b0011_1001
// reverse get data
int newValue3 = result & 0b000_0011;
// shiting data to 2 digits right
result = result >> 2;
int newValue2 = result & 0b000_0011;
// shifting data to 2 digits right
result = result >> 2;
int newValue1 = result & 0b000_0011;
GD.Print(newValue1);    // 3
GD.Print(newValue2);   // 2
GD.Print(newValue3);   // 1
```

We retrieve the numbers in the reverse order they were stored. Since we know that each stored number takes up only two digits, we essentially only need to retrieve the last two bits. To do this, we use the bit mask 0b000_0011 and the logical multiplication operation, which returns 1 if each of the two corresponding digits is 1. That is, the operation

```
int newValue3 = result & 0b000_0011;
```

Equivalent to:

```
0b0011_1001
*
0b0000_0011
=
0b0000_0001
```

Thus, the last number is 0b0000_0001, or 1 in decimal.

It's worth noting that if we know the exact structure of the data, we can easily create a bitmask to obtain the desired number:

```
result = 0b0011_1001;
int recreatedValue1 = (result & 0b0011_0000) >> 4;
GD.Print(recreatedValue1);
```

Here we get the first number, which, as we know, occupies bits 4 and 5 in the number. To do this, we multiply by the bit mask 0b0011_0000. And then we shift the number 4 places to the right.

```
0b0011_1001
*
0b0011_0000
```

```
=
0b0011_0000
>> 4
=
0b0000_0011

```

Similarly, if we know exactly the structure in which the data is stored, we could store the data directly in the right place in the result number:

```
int value1 = 3; // 0b0000_0011
int value2 = 2; // 0b0000_0010
int value3 = 1; // 0b0000_0001
int result = 0b0000_0000;
// saving in result values from value1
result = result | (value1 << 4);
// saving in result values from value2
result = result | (value2 << 2);
// saving in result values from value3
result = result | value3; // 0b0011_1001

GD.Print(result); // 57
```

Assignment operations

Assignment operators assign a value. Assignment operators involve two operands, with the left operand being limited to a named expression being modified, such as a variable.

Like many other programming languages, C# has a basic assignment operator, =, which assigns the value of the right operand to the left operand:

```
1. int number = 23;
```

Here, the variable number is assigned the number 23. The variable number represents the left operand, which is assigned the value of the right operand, which is the number 23.

You can also perform multiple assignments to multiple variables simultaneously:

```
1. int a, b, c;
2. a = b = c = 34;
```

It's worth noting that assignment operations have low precedence. The value of the right operand will be calculated first, and only then will that value be assigned to the left operand. For example:

```
1. int a, b, c;
2. a = b = c = 34 * 2 / 4; // 17
```

First, the expression 34 * 2 / 4 will be evaluated, then the resulting value will be assigned to variables.

Besides the basic assignment operation, C# has a number of other operations:

- **+=**: assignment after addition. Assigns the sum of the left and right operands to the left operand: A += B is equivalent to A = A + B
- **-=**: assignment after subtraction. Assigns the difference of the left and right operands to the left operand: A -= B is equivalent to A = A - B
- ***=**: assignment after multiplication. Assigns the product of the left and right operands to the left operand: A *= B is equivalent to A = A * B
- **/=**: assignment after division. Assigns the quotient of the left and right operands to the left operand: A /= B is equivalent to A = A / B
- **%=**: assignment after modulo division. Assigns the remainder of the integer division of the left operand by the right operand to the left operand: A %= B is equivalent to A = A % B
- **&=**: assignment after bitwise conjunction. Assigns to the left operand the result of the bitwise conjunction of its bit representation with the bitwise representation of the right operand: A &= B is equivalent to A = A & B
- **|=**: assignment after bitwise disjunction. Assigns to the left operand the result of the bitwise disjunction of its bit representation with the bitwise representation of the right operand: A |= B is equivalent to A = A | B
- **^=**: assignment after exclusive OR. Assigns to the left operand the result of the exclusive OR of its bit representation with the bitwise representation of the right operand: A ^= B is equivalent to A = A ^ B
- **<<=**: assignment after left shift. Assigns to the left operand the result of shifting its bitwise representation left by a certain number of bits equal to the value of the right operand: A <<= B is equivalent to A = A << B
- **>>=**: assignment after right shift. Assigns to the left operand the result of shifting its bit representation to the right by a certain number of bits, equal to the value of the right operand: A >>= B is equivalent to A = A >> B

Application of assignment operations:

```
1. int a = 10;
2. a += 10;        // 20
3. a -= 4;         // 16
4. a *= 2;         // 32
5. a /= 8;         // 4
6. a <<= 4;      // 64
7. a >>= 2;      // 16
```

Assignment operations are right-associative, meaning they are performed from right to left. For example:

```
1. int a = 8;
2. int b = 6;
3. int c = a += b -= 5;    // 9
```

In this case, the expression will be executed as follows:

1. b -= 5 (6-5=1)
2. a += (b-=5) (8+1 = 9)
3. c = (a += (b-=5)) (c = 9)

Conversions of basic data types

When discussing data types, we specified the values each type can hold and how many bytes of memory it can occupy. In the previous topic, we looked at arithmetic operations. Now let's apply the addition operation to data of different types:

```
1. byte a = 4;
2. int b = a + 70;
```

The result of the operation is quite rightly the number 74, as expected.

But now let's try adding two objects of type **byte:**

```
1. byte a = 4;
2. byte b = a + 70;  // error
```

Here, only the type of the variable receiving the result of the addition has changed—from int to byte. However, when we try to compile the program, we get a compile-time error. And if we're working in Visual Studio/VS Code, the environment will underline the second line with a red line, indicating an error.

When performing operations, we must consider the range of values that a given type can store. But in this case, the number 74 we expect to obtain is well within the range of values for the byte type, yet we still get an error.

The point is that the addition operation (and subtraction) returns an int value if the operation involves integer data types with a bit depth less than or equal to int (that is, byte, short, and int). Therefore, the result of the operation a + 70 is an object with a memory length of 4 bytes. We then attempt to assign this object to the variable b, which is of type byte and occupies 1 byte of memory.

To resolve this situation, we need to apply a type conversion operation. The type conversion operation requires specifying in parentheses the type to which the value should be converted:

```
1. (data_type_to_be_converted)value_to_be_converted;
```

So, let's modify the previous example by applying the type conversion operation:

```
1. byte a = 4;
2. byte b = (byte)(a + 70);
```

Narrowing and expanding transformations

Transformations can be narrowing or widening. Widening transformations expand the size of an object in memory. For example:

```
1. byte a = 4;              // 0000100
2. ushort b = a;    // 000000000000100
```

In this case, a variable of type ushort is assigned a value of type byte. The byte type occupies 1 byte (8 bits), and the value of variable a in binary form can be represented as:

```
1. 00000100
```

A ushort value takes up 2 bytes (16 bits). When assigned to variable b, the value of variable a expands to 2 bytes.

```
1. 0000000000000100
```

So, value, that occupies 8 bits, is **expanded** to 16 bits.

Narrowing conversions, on the other hand, narrow a value to a lower-bit type. In the second listing of the article, we dealt with narrowing conversions:

```
1. ushort a = 4;
2. byte b = (byte) a;
```

Here, the variable b, which occupies 8 bits, is assigned the value ushort, which occupies 16 bits. That is, from 00000000000000100 we get 00000100. Thus, the value is narrowed from 16 bits (2 bytes) to 8 bits (1 byte).

Explicit and implicit transformations

Implicit Conversions

In the case of widening conversions, the compiler performed all data conversions for us; that is, the conversions were implicit. Such conversions don't present any particular difficulties. However, it's worth mentioning the general mechanics of such conversions.

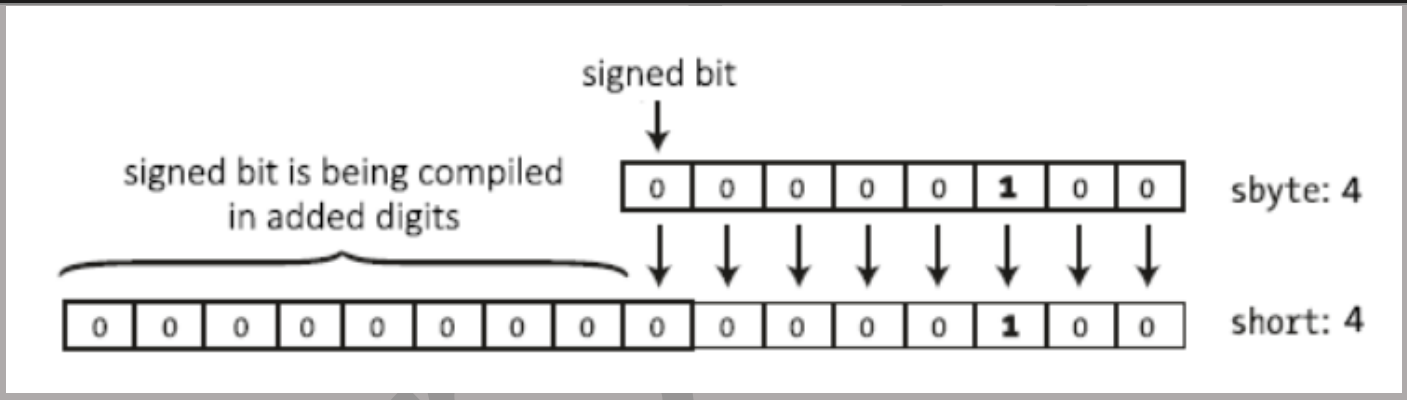
When converting from an unsigned type of lower bit width to an unsigned type of higher bit width, extra bits are added that have the value 0. This is called **zero extension**.

```
1. byte a = 4;              // 0000100
2. ushort b = a;    // 000000000000100
```

When converting to a signed type, the bit representation is padded with zeros if the number is positive and with ones if the number is negative. The last digit of the number contains the sign bit—0 for positive numbers and 1 for negative numbers. When expanding, the sign bit is compiled into the padded digits.

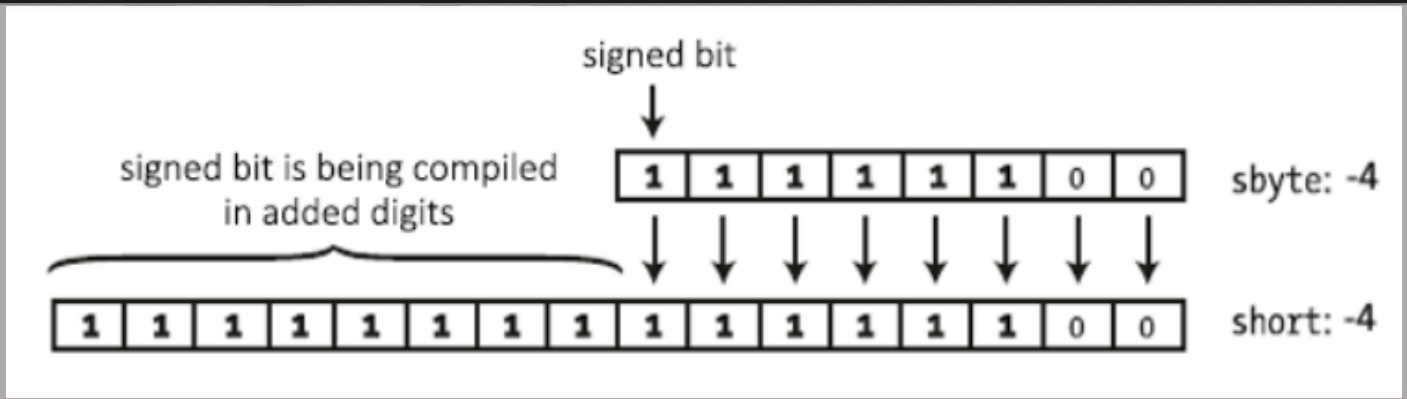
Let's consider the conversion of a positive number:

```
1. sbyte a = 4;             // 0000100
2. short b = a;     // 000000000000100
```



Converting a negative number:

```
1. sbyte a = -4;            // 1111100
2. short b = a;     // 111111111111100
```



Explicit conversion

With **explicit conversions**, we must apply the conversion operation (the () operator) ourselves. The essence of the type conversion operation is that the value is preceded by the type to which it should be converted in parentheses:

```
1. int a = 4;
2. int b = 6;
3. byte c = (byte)(a+b);
```

The compiler implicitly performs widening conversions from a type with a smaller bit depth to a type with a larger bit depth. These conversion chains can include the following:

byte -> **short** -> **int** -> **long** -> **decimal**

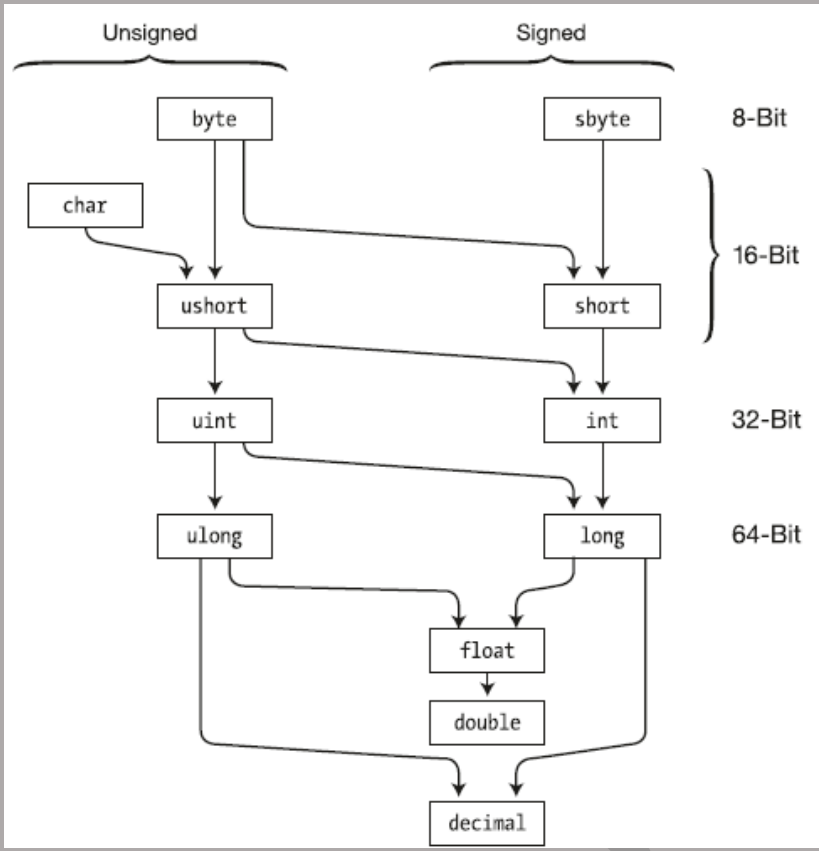
int -> **double**

short -> **float** -> **double**

char -> **int**

All safe automatic transformations can be described by the following table:

Type	**Types, that are safe to convert**
byte	short, ushort, int, uint, long, ulong, float, double, decimal
sbyte	short, int, long, float, double, decimal
short	int, long, float, double, decimal
ushort	int, uint, long, ulong, float, double, decimal
int	long, float, double, decimal
uint	long, ulong, float, double, decimal
long	float, double, decimal
ulong	float, double, decimal
float	double
char	ushort, int, uint, long, ulong, float, double, decimal



In other cases, explicit type conversions should be used.

It should also be noted that although both double and decimal can store fractional data, and decimal has a larger bit depth than double, a double value must still be explicitly cast to decimal:

```
1. double a = 4.0;
2. decimal b = (decimal)a;
```

Loss of data accuracy

Let's consider another situation, such as the following:

```
1. int a = 33;
2. int b = 600;
3. byte c = (byte)(a+b);
4. GD.Print(c);   // 121
```

The result will be 121, since 633 is outside the valid range for the byte type, and the high-order bits will be truncated. The final result will be 121. Therefore, this must be taken into account when converting. In this case, we can either choose numbers a and b that add up to no more than 255, or we can choose a different data type instead of byte, such as int.

Conditional expressions

A separate set of operations represents conditional expressions. These operations return a logical value, that is, a **bool** value: **true** if the expression is true, and **false** if the expression is false. These operations include comparison and logical operators.

Comparison Operators

Comparison operators compare two operands and return a **bool** value—**true** if the expression is true and **false** if the expression is false.

- **==**
  Compares two operands for equality. If they are equal, the operator returns true; if they are not equal, it returns false:

```
1. int a = 10;
2. int b = 4;
3. bool c = a == b; // false
```

- **!=**
  Compares two operands and returns true if they are not equal, and false if they are equal.

```
1. int a = 10;
2. int b = 4;
3. bool c = a != b;     // true
4. bool d = a!=10;      // false
```

- **<**
  Less-than operator. Returns true if the first operand is less than the second, and false if the first operand is greater than the second:

```
1. int a = 10;
2. int b = 4;
3. bool c = a < b; // false
```

- **>**
  Greater-than operator. Compares two operands and returns true if the first operand is greater than the second, otherwise it returns false:

```
1. int a = 10;
2. int b = 4;
3. bool c = a > b;      // true
4. bool d = a > 25;     // false
```

- **<=**
  Less-than or equal operator. Compares two operands and returns true if the first operand is less than or equal to the second. Otherwise, it returns false.

```
1. int a = 10;
2. int b = 4;
3. bool c = a <= b;      // false
4. bool d = a <= 25;     // true
```

- **>=**
  Greater-than or equal operator. Compares two operands and returns true if the first operand is greater than or equal to the second, otherwise it returns false:

```
1. int a = 10;
2. int b = 4;
```

```
3. bool c = a >= b;      // true
4. bool d = a >= 25;     // false
```

The operators <, > <=, >= have higher precedence than == and !=.

Logical Operators

C# also defines logical operators, which also return a bool value. They accept bool values as operands. They are typically applied to relations and combine multiple comparison operations.

- **|**

  Logical addition, or logical OR, operator. Returns true if at least one of the operands is true.

```
1. bool x1 = (5 > 6) | (4 < 6); // 5 > 6 is false, 4 < 6 is true, so true is returned
2. bool x2 = (5 > 6) | (4 > 6); // 5 > 6 is false, 4 > 6 is false, so false is returned
```

- **&**

  Logical multiplication, or logical AND, operator. Returns true if both operands are simultaneously true.

```
1. bool x1 = (5 > 6) & (4 < 6); // 5 > 6 is false, 4 < 6 is true, so false is returned
2. bool x2 = (5 < 6) & (4 < 6); // 5 < 6 is true, 4 < 6 is true, so true is returned
```

- **||**

  Logical addition operator. Returns true if at least one of the operands is true.

```
1. bool x1 = (5 > 6) || (4 < 6); // 5 > 6 is false, 4 < 6 is true, so true is returned
2. bool x2 = (5 > 6) || (4 > 6); // 5 > 6 is false, 4 > 6 is false, so false is returned
```

- **&&**

  Logical multiplication operator. Returns true if both operands are simultaneously true.

```
1. bool x1 = (5 > 6) && (4 < 6); // 5 > 6 is false, 4 < 6 is true, so false is returned
2. bool x2 = (5 < 6) && (4 < 6); // 5 < 6 is true, 4 < 6 is true, so true is returned
```

- **!**

  Logical negation operator. Performed on one operand and returns true if the operand is false. If the operand is true, the operation returns false:

```
1. bool a = true;
2. bool b = !a; // false
```

- **^**

  Exclusive OR operator. Returns true if either the first or second operand (but not both) is true; otherwise, it returns false.

```
1. bool x5 = (5 > 6) ^ (4 < 6); // 5 > 6 is false, 4 < 6 is true, so true is returned
2. bool x6 = (50 > 6) ^ (4 / 2 < 3); // 50 > 6 is true, 4 / 2 < 3 is true, so false is returned
```

Here, we have two pairs of operations **|** and **||** (as well as **&** and **&&**) that perform similar actions, but they are not equivalent.

In the expression **z=x|y;** both x and y will be evaluated.

In the expression **z=x||y;** x will be evaluated first, and if it is **true**, then evaluating y is pointless, since z will already be **true** anyway. y will only be evaluated if x is **false**.

The same applies to the pair of operations **&/&&**. In the expression **z=x&y;** both x and y will be evaluated.

In the expression **z=x&&y;** x will be evaluated first, and if it is **false**, then evaluating y is pointless, since z will already be **false** anyway. y will only be evaluated if x is **true**.

Therefore, the operations **||** and **&&** are more convenient for calculations, as they reduce the time it takes to evaluate an expression, thereby improving performance. The **|** and **&** operators, however, are better suited for performing bitwise operations on numbers.

14.1.4 Statements

If..else construct and the ternary operator

Conditional constructs are a fundamental component of many programming languages, directing program execution along a specific path based on certain conditions. One such construct in the C# programming language is the if..else construct.

The if/else construct checks the truth of a condition and executes specific code based on the results of the check.

Its simplest form consists of an if block:

```
1. if(condition)
2. {
3.     functions to execute
4. }
```

The if keyword is followed by a condition. The condition must represent a bool value. This can be a bool value itself or the result of a conditional expression or another expression that returns a bool value. If this condition is true, the code placed after the condition within the curly braces is executed.

```
1. int num1 = 8;
2. int num2 = 6;
3. if(num1 > num2)
4. {
5.     GD.Print($"The number {num1} is greater than the number {num2}");
6. }
```

In this case, the first number is greater than the second, so the expression num1 > num2 is true and returns true, and control passes to the line GD.Print("The number {num1} is greater than the number {num2}");

If the if block contains a single statement, we can shorten it by removing the curly braces:

```
1. int num1 = 8;
2. int num2 = 6;
3. if (num1 > num2)
4.     GD.Print($"The number {num1} is greater than the number {num2}");
5.
6. // or
7. if (num1 > num2) GD.Print($"The number {num1} is greater than the number {num2}");
```

We can also combine several conditions at once using logical operators:

```
1. int num1 = 8;
2. int num2 = 6;
3. if(num1 > num2 && num1==8)
4. {
5.     GD.Print($"The number {num1} is greater than the number {num2}");
6. }
```

In this case, the if block will be executed if num1 > num2 is true and num1==8 is true.

Else Statement

But what if we want to also perform certain actions if the condition is not met? In this case, we can add an else block:

```
int num1 = 8;
int num2 = 6;
if(num1 > num2)
{
    GD.Print($"The number {num1} is greater than the number {num2}");
}
else
{
    GD.Print($"The number {num1} is less than or equals the number {num2}");
}
```

The else block is executed if the condition after the if statement is false. If the else block contains only one statement, we can shorten it by removing the curly braces:

```
int num1 = 8;
int num2 = 6;
if(num1 > num2)
    GD.Print($"The number {num1} is greater than the number {num2}");
else
    GD.Print($"The number {num1} is less or equals the number {num2}");
```

else if

But in the example above, when comparing numbers, we can count three possible conditions: the first number is greater than the second, the first number is less than the second, and the numbers are equal. Using the else if construct, we can handle additional conditions:

```
int num1 = 8;
int num2 = 6;
if(num1 > num2)
{
    GD.Print($"The number {num1} is greater than the number {num2}");
}
else if (num1 < num2)
{
    GD.Print($"The number {num1} is less than the number {num2}");
}
else
{
    GD.Print("The number num1 equals the number num2");
}

```

You can add multiple else if statements if needed:

```
string name = "Alex";

if (name == "Tom")
    GD.Print("Your name is Tomas");
else if (name == "Bob")
    GD.Print("Your name is Robert");
else if (name == "Mike")
    GD.Print("Your name is Michael");
else
    GD.Print("Unknown name");

```

Ternary Operator

A ternary operator also allows you to test a condition and perform certain actions based on its truth value. It has the following syntax:

```
1. [first operand - condition] ? [second operand] : [third operand]
```

There are three operands here. Depending on the condition, the ternary operator returns the second or third operand: if the condition is true, the second operand is returned; if the condition is false, the third. For example:

```
1. int x=3;
2. int y=2;
3.
4. int z = x < y ? (x + y) : (x - y);
5. GD.Print(z);   // 1
```

Here, the first operand (i.e., the condition) represents the expression **x < y**. If it evaluates to **true**, the second operand, **(x + y)**, is returned, which is the result of the addition operation. If the condition evaluates to **false**, the third operand, **(x - y)**, is returned.

The result of the ternary operation (i.e., the second or third operand, depending on the condition) is assigned to the variable **z**.

14.1.5 Loops

Loops are control constructs that allow a given action to be performed multiple times, depending on certain conditions. C# has the following types of loops:

- for
- foreach
- while
- do...while

For Loop

The for loop has the following formal definition:

```
for ([actions_before_loop_execution]; [condition]; [actions_after_execution])
{
    // actions
}
```

A for loop declaration consists of three parts. The first part of the loop declaration is some actions that are performed once before the loop is executed. This is typically where variables that will be used in the loop are defined.

The second part is the condition under which the loop will execute. As long as the condition is true, the loop will execute.

And the third part is some actions that are performed after the loop block completes. These actions are performed each time the loop block completes.

After the loop declaration, the loop actions themselves are placed in curly braces.

Let's consider a standard for loop:

```
for (int i = 1; i < 4; i++)
{
    GD.Print(i);
}
```

Here, the first part of the loop declaration—int i = 1—creates and initializes the variable i.

The second part is the condition i < 4. That is, as long as the variable i is less than 4, the loop will execute.

And the third part is the actions performed after the actions in the loop block complete—incrementing the variable i by one.

The entire loop process can be represented as follows:

1. The variable int i = 1 is defined.
2. The condition i < 4 is checked. It is true (since 1 is less than 4), so the loop block, specifically the GD.Print(i) statement, is executed, displaying the value of the variable i to the console.
3. The loop block has finished executing, so the third part of the loop declaration, i++, is executed. After this, the variable i will be equal to 2.
4. The condition i < 4 is checked again. It is true (since 2 is less than 4), so the loop block, GD.Print(i), is executed again.

5. The loop block has finished executing, so the i++ expression is executed again. After this, the variable i will be equal to 3.
6. The condition i < 4 is checked again. It is true (since 3 is less than 4), so the loop block, GD.Print(i), is executed again.
7. The loop block has finished executing, so the i++ expression is executed again. After this, the variable i will be equal to 4.
8. The condition i < 4 is checked again. This time, it returns false, since the value of the variable i is NOT less than 4, so the loop terminates. The rest of the program that follows the loop is then executed.

As a result, the loop block will execute three times until the value of i equals 4. Each time, this value will be incremented by 1. A single execution of the loop block is called an iteration. Therefore, the loop here will execute three iterations. The console output:

```
Godot Engine v4.6.1.stable.mono.official.14d19694e - https://godotengine.org
Vulkan 1.4.325 - Forward Mobile - Using Device #0: NVIDIA - NVIDIA GeForce RTX 4060 Ti

1
2
3
--- Debugging process stopped ---
```

If the for loop block contains a single statement, we can shorten it by removing the curly braces:

```
1. for (int i = 1; i < 4; i++)
2.     GD.Print(i);
3.
4. // or
5. for (int i = 1; i < 4; i++) GD.Print(i);
```

It's not necessary to declare a variable in the first part of the loop and change its value in the third part—these actions can be anything. For example:

```
1. var i = 1;
2.
3. for (GD.Print("Loop execution start"); i < 4; GD.Print($"i = {i}"))
4. {
5.     i++;
6. }
```

Here again, the loop runs while the variable i is less than 4, but the increment of the variable i occurs within the loop block. The console output of this program:

```
Godot Engine v4.6.1.stable.mono.official.14d19694e - https://godotengine.org
Vulkan 1.4.325 - Forward Mobile - Using Device #0: NVIDIA - NVIDIA GeForce RTX 4060 Ti

Loop execution start
i = 2
i = 3
i = 4
--- Debugging process stopped ---
```

We don't have to specify all the conditions when declaring a loop. For example, we could write it like this:

```
1. int i = 1;
2. for (; ;)
3. {
4.     GD.Print($"i = {i}");
5.     i++;
6. }
```

Formally, the loop definition remains the same, only now the blocks in the definition are empty: for (; ;). We have no initialized variable and no condition, so the loop will run forever —an infinite loop.

We can also omit some blocks:

```
1. int i = 1;
2. for (; i<4;)
3. {
4.     GD.Print($"i = {i}");
5.     i++;
6. }
```

This example is essentially equivalent to the first example: we also have a counter variable, only it's defined outside the loop. We have a loop execution condition. And we increment the variable within the for block itself.

It's also worth noting that you can define multiple variables within a loop declaration:

```
1. for (int i = 1, j = 1; i < 10; i++, j++)
2.     GD.Print($"{i * j}");
```

Here, the first part of the loop declaration defines two variables: **i** and **j**. The loop executes until **i** equals 10. After each iteration, the variables **i** and **j** are incremented by one. The program's console output:

```
Godot Engine v4.6.1.stable.mono.official.14d19694e - https://godotengine.org
Vulkan 1.4.325 - Forward Mobile - Using Device #0: NVIDIA - NVIDIA GeForce RTX 4060 Ti

1
4
9
16
25
36
49
64
81
--- Debugging process stopped ---
```

Do…While Loop

14.1.6 Arrays

14.1.7 Collections

14.1.8 Functions/Methods

14.1.9 Switch Construction

14.1.10 Enumeration Types

14.1.10 Classes

14.1.11 Constructors

14.1.12 Structs

14.1.13 Namespaces

14.1.14 Access Modifiers

14.1.15 Properties

14.1.16 Method Overloading

14.1.17 Static Modifier

14.1.18 Nullable Types

14.1.19 Object Oriented Programming

Inheritance

Polymorphism

Encapsulation

14.2 Intermediate Topic

This topic includes tutorials/explanations of intermediate difficulty for developers, who have some kind of good experience in C# coding. Before you try to tackle with working on intermediate materials, firstly recommended to get some basic knowledge in C# coding: try to make, for example, generics class or extension methods not directly in Godot Engine, instead in Visual Studio console application.

This “intermediate topic” includes such themes:

- Generics
- Interfaces
- Extension Methods

14.2.1 Generics

In addition to standard types, the .NET framework also supports generic types and the creation of generic methods. To understand the specifics of this phenomenon, let's first look at a problem that could arise before the advent of generic types. Generics is a powerful C# feature that allows to write type-safe, reusable code without being tied to a specific type. In the context of Godot 4.6, this is especially useful for game systems.

The angle brackets in the class or method <T> indicates that the class or method is generic, and the type T enclosed in the angle brackets will be used by this class or method. The letter T doesn't have to be used; it could be any other letter or character set. Moreover, at the coding stage, we don't know what type this will be; it could be any type. Therefore, the T parameter enclosed in angle brackets is also called a generic parameter, since any type can be substituted for it.

As an examples, we will try to write:

- a **simple example** with generic class Person<T>
- an **extension** for comfortable work with node tree in Godot, which will have two static methods: first method finds first child node of type T, second method finds all child nodes of type T.

A simple example with generic class Person<T>

```
1. public class Person<T>
2. {
3.     public T Id { get; set; }
4.     public string Name { get; set; }
5.
6.     public Person (T id, string name)
7.     {
8.         Id = id;
9.         Name = name;
10.    }
11. }
```

Defining generic class Person<T>, with generic parameter <T>Id

```
1. using Godot;
2.
3. public partial class PersonManager : Node
4. {
5.     public override void _Ready()
6.     {
7.         Person<int> bob = new Person<int>(123, "Bob");
8.         Person<string> tom = new Person<string>("id3", "Tom");
9.
10.        GD.Print($"Person Id: {bob.Id}, Person Name: {bob.Name}");
11.        GD.Print($"Person Id: {tom.Id}, Person Name: {tom.Name}");
12.    }
13. }
```

Creating objects of class Person<T> and printing values

A Node extension example

In this example there are implementations of static generic methods: FindChild<T>, FindAllChidren<T> and CollectChildren<T>.

```
using Godot;
using System.Collections.Generic;

namespace Generics
{
    public static class NodeExtensions
    {
        public static T FindChild<T>(this Node parent) where T : Node
        {
            foreach (Node child in parent.GetChildren())
            {
                if (child is T found)
                    return found;

                T result = child.FindChild<T>();
                if (result != null)
                    return result;
            }
            return null;
        }

        public static List<T> FindAllChildren<T>(this Node parent) where T : Node
        {
            List<T> results = new();
            CollectChildren(parent, results);
            return results;
        }

        private static void CollectChildren<T>(Node node, List<T> list) where T : Node
        {
            foreach (Node child in node.GetChildren())
            {
                if (child is T match)
                    list.Add(match);
                CollectChildren(child, list);
            }
        }
    }
}
```

Static class NodeExtensions

```
using Godot;

namespace Generics
{
    public partial class GameManager : Node
    {
        public override void _Ready()
        {
            Camera2D cam = this.FindChild<Camera2D>();
            GD.Print($"Camera: {cam}");

            var enemies = GetTree().Root.FindAllChildren<Enemy>();
            GD.Print($"Found enemies: {enemies.Count}");
        }
    }
}

```

Calling NodeExtensions class methods

So, in summary, Generics are allowing to:

- **Reuse logic** (pools, state machines, search) for any node type
- **Catching errors** at compile time, not at runtime
- **Improve readability** – developer can immediately see what type of the code he is working with
- **Avoid boxing/unboxing** – less issues with performance

14.2.2 Interfaces

An interface represents a reference type that can define certain functionality—a set of methods and properties without implementation. This functionality is then implemented by classes and structures that implement these interfaces.

Defining an Interface

The interface keyword is used to define an interface. Interface names in C# typically begin with a capital I, for example, IComparable, IEnumerable (the so-called Hungarian notation), but this is not a requirement, but rather a programming style.

What can an interface define? In general, interfaces can define the following entities:

- Methods
- Properties
- Indexers
- Events
- Static fields and constants (since C# 8.0)

Like classes, interfaces have internal access by default, meaning they are only accessible within the current project. However, using the public modifier, we can make an interface publicly accessible:

```
1. public interface IInteractable
2. {
3.     void Interact();
4. }
```

Also, starting with C# 8.0, interfaces support default implementations of methods and properties. This means we can define full-fledged methods and properties in interfaces that have implementations similar to those in regular classes and structures. For example, let's define a default implementation of the Interact method:

```
1. interface IInteractable
2. {
3.     void Interact()
4.     {
5.         GD.Print("Interacting");
6.     }
7. }
```

Using interfaces

An interface represents a type description, a set of components that must have a data type. And, in fact, we can't create interface objects directly using a constructor, as we can in classes.

Ultimately, an interface is intended to be implemented in classes and structures. For example, let's implement the ISelectable interface defined above:

```
1. namespace InterfacesTraining
2. {
3.     public interface ISelectable
4.     {
5.         void OnSelect();
6.     }
7. }
```

ISelectable interface

```
 1. using Godot;
 2.
 3. namespace InterfacesTraining
 4. {
 5.     public partial class BoxController : Area2D, ISelectable
 6.     {
 7.         public void OnSelect()
 8.         {
 9.             GD.Print($"[Box] Selected: {Name}, position: {GlobalPosition}");
10.         }
11.     }
12. }
```

ISelectable interface definition in BoxController

```
 1. using Godot;
 2.
 3. namespace InterfacesTraining
 4. {
 5.     public partial class CircleController : Area2D, ISelectable
 6.     {
 7.         public void OnSelect()
 8.         {
 9.             GD.Print($"[Box] Selected: {Name}, position: {GlobalPosition}");
10.         }
11.     }
12. }
```

ISelectable interface definition in CircleController

```cs
using Godot;

namespace InterfacesTraining
{
    public partial class SelectionManager : Node2D
    {
        public override void _UnhandledInput(InputEvent @event)
        {
            if (@event is InputEventMouseButton mouseEvent
                && mouseEvent.ButtonIndex == MouseButton.Left
                && mouseEvent.Pressed)
            {
                // Convert screen coordinates to world coordinates
                Vector2 mouseWorldPosition = GetCanvasTransform().AffineInverse() * mouseEvent.Position;

                GD.Print($"[Debug] Click in world coordinates: {mouseWorldPosition}");

                CheckSelection(mouseWorldPosition);
            }
        }

        private void CheckSelection(Vector2 mousePosition)
        {
            var spaceState = GetWorld2D().DirectSpaceState;

            var query = new PhysicsPointQueryParameters2D
            {
                Position = mousePosition,
                CollideWithAreas = true,
                CollideWithBodies = false
            };

            var results = spaceState.IntersectPoint(query);

            GD.Print($"[Debug] Intersections found: {results.Count}");

            foreach (var result in results)
            {
                GD.Print($"[Debug] Collider: {result["collider"].AsGodotObject()}");

                if (result["collider"].AsGodotObject() is ISelectable selectable)
                {
                    selectable.OnSelect();
                }
            }
        }
    }
}
```

Using Interface in Program

Scene Structure

```
MainScene (Node2D)
├── SelectionManager (Node)            ← SelectionManager.cs
├── BoxController (Area2D)             ← BoxController.cs
│   ├── Sprite2D
│   └── CollisionShape2D               ← RectangleShape2D
└── CircleController (Area2D)          ← CircleController.cs
    ├── Sprite2D
    └── CollisionShape2D               ← CircleShape2D
```

This program defines a CheckSelection() method that calls an ISelectable interface method once user tries to click BoxController or CircleController.

```
if (result["collider"].AsGodotObject() is ISelectable selectable)
{
    selectable.OnSelect();
}
```

In this case: if one of selected object have a definition of ISelectable interface, then it calls method OnSelect() of clicked controller.

In other words, an interface is a contract that a certain type necessarily implements a certain functionality.

Key Points:

- **ISelectable is selectable** – is a pattern matching method from C#. It simultaneously checks the type and casts the object, allowing you to call an interface method without knowing the specific class.
- **CollideWithAreas = true** is required, otherwise Area2Ds won't be found (by default, only bodies are searched).
- **_UnhandledInput** is used instead of _Input to give UI elements (buttons, etc.) priority and allow them to "absorb" clicks.

14.2.3 IEnumerable and IEnumerator Interfaces

14.2.4 Delegates

14.2.5 Events

14.2.6 Lambda Expressions and Anonymous Functions

14.2.7 Exception Handling

14.2.X Extension Methods

Type extensions allow you to add new components—methods, properties, and operators—to existing types without creating a new derived type, recompiling, or otherwise modifying the original type. This is even true if you don't have access to the type's source code. Two types of syntax are used for type extensions: extension methods and extension blocks. In our case, we need to use only extension methods, extension blocks are not so necessary to use.

Extension methods allow you to add new methods to existing types without creating a new derived class. This functionality is especially useful when we want to add a new method to a type, but we can't modify the type itself (class or struct) because we don't have access to the source code. Or when we can't use the standard inheritance mechanism, for example, if classes are defined with the sealed modifier.

Benefits of using Extension Methods:

- **Readability:** Code becomes more natural and consistent
- **Organization:** Groups related utilities in one place
- **Reusability:** Write once, use everywhere
- **Doesn't change source code:** No need to extend Godot classes

Important rules:

- Extension methods must be in a **static class**
- The first parameter must contain the **this** keyword
- The class must be in **global scope** (not nested)
- After adding extensions, you must **rebuild** the project in Godot

These examples will help make your Godot code cleaner and easier to read!

```gdscript
public static class VectorExtensions
{
    // Check if Vector is within distance
    public static bool IsWithinDistance(this Vector2 from, Vector2 to, float distance)
    {
        return from.DistanceTo(to) <= distance;
    }

    // Random vector in radius
    public static Vector2 RandomInRadius(this Vector2 center, float radius)
    {
        var angle = GD.RandRange(0, Mathf.Tau);
        var distance = GD.RandRange(0, radius);
        return center + new Vector2(Mathf.Cos(angle), Mathf.Sin(angle)) * distance;
    }

    // Get direction to target
    public static Vector2 DirectionTo(this Vector2 from, Vector2 to)
    {
        return (to - from).Normalized();
    }

    // Clamp vector length
    public static Vector3 ClampLength(this Vector3 vector, float minLength, float maxLength)
    {
        var length = vector.Length();
        if (length < minLength)
            return vector.Normalized() * minLength;
        if (length > maxLength)
            return vector.Normalized() * maxLength;
        return vector;
    }
}
// var isClose = position.IsWithinDistance(target, 10.0f);
// var randomPos = center.RandomInRadius(50.0f);
// var direction = playerPos.DirectionTo(enemyPos);
```

An extension for Vector2/Vector3

```gdscript
public static class ColorExtensions
{
    // Create color with a new alpha
    public static Color WithAlpha(this Color color, float alpha)
    {
        return new Color(color.R, color.G, color.B, alpha);
    }

    // Lighten color
    public static Color Lighten(this Color color, float amount)
    {
        return color.Lightened(amount);
    }

    // Random color
    public static Color Random(this Color _)
    {
        return new Color(
            (float)GD.RandRange(0, 1),
            (float)GD.RandRange(0, 1),
            (float)GD.RandRange(0, 1)
        );
    }
}

// Usage:
// var semiTransparent = Colors.Red.WithAlpha(0.5f);
// var lighter = originalColor.Lighten(0.3f);
// var randomColor = new Color().Random();
```

An extension for Color

```gdscript
public static class AreaExtensions
{
    // Get all bodies of type T in area
    public static List<T> GetOverlappingBodiesOfType<T>(this Area2D area) where T : Node2D
    {
        var result = new List<T>();
        foreach (var body in area.GetOverlappingBodies())
        {
            if (body is T typedBody)
                result.Add(typedBody);
        }
        return result;
    }

    // Check, if there is a body of type T in area
    public static bool HasOverlappingBodyOfType<T>(this Area2D area) where T : Node2D
    {
        foreach (var body in area.GetOverlappingBodies())
        {
            if (body is T)
                return true;
        }
        return false;
    }
}

// Usage:
// var enemies = detectionArea.GetOverlappingBodiesOfType<Enemy>();
// if (pickupArea.HasOverlappingBodyOfType<Player>()) { }
```

An extension for Area2D/Area3D

```gdscript
public static class InputExtensions
{
    // Get a vector from input
    public static Vector2 GetMovementVector(string up, string down, string left, string right)
    {
        return new Vector2(
            Input.GetActionStrength(right) - Input.GetActionStrength(left),
            Input.GetActionStrength(down) - Input.GetActionStrength(up)
        ).Normalized();
    }

    // Check buttons combination
    public static bool IsActionCombo(params string[] actions)
    {
        foreach (var action in actions)
        {
            if (!Input.IsActionPressed(action))
                return false;
        }
        return true;
    }
}

// Usage:
// var movement = InputExtensions.GetMovementVector("move_up", "move_down", "move_left", "move_right");
// if (InputExtensions.IsActionCombo("ctrl", "shift", "save")) { }
```

An extension for Input

14.3 Advanced Topic

14.3.1 LINQ

14.3.2 Multithreading

14.3.3 Asynchronous Programming

14.3.4 Dynamic Language Runtime

14.3.5 Reflection

14.X Extra Topic

14.X.1 C# API Differences to GDScript

General differences

PascalCase is used to access Godot APIs in C# instead of the snake_case used by GDScript and C++. Where possible, fields and getters/setters have been converted to properties. In general, the C# Godot API strives to be as idiomatic as is reasonably possible.

In GDScript, the setters/getters of a property can be called directly, although this is not encouraged. In C#, only the property is defined. For example, to translate the GDScript code `x.set_name("Friend")` to C#, write `x.Name = "Friend";`.

A C# IDE will provide IntelliSense, which is extremely useful when figuring out renamed C# APIs. The built-in Godot script editor has no support for C# IntelliSense, and it also doesn't provide many other C# development tools that are considered essential.

Global scope

Global functions and some constants had to be moved to classes, since C# does not allow declaring them in namespaces. Most global constants were moved to their own enums.

Constants

In C#, only primitive types can be constant. For example, the `TAU` constant is replaced by the `Mathf.Tau` constant, but the `Vector2.RIGHT` constant is replaced by the `Vector2.Right` read-only property. This behaves similarly to a constant, but can't be used in some contexts like `switch` statements.

Global enum constants were moved to their own enums. For example, `ERR_*` constants were moved to the `Error` enum.

14.X.X Mobile Input

14.X.X PC Input

14.X.X Console Input

14.X Design Patterns

14.X.1 State Machine Pattern

15 C++ Godot

16 Settings

16.1 Project Settings

16.2 Editor Settings

17 Shaders

18 User Interface (UI)

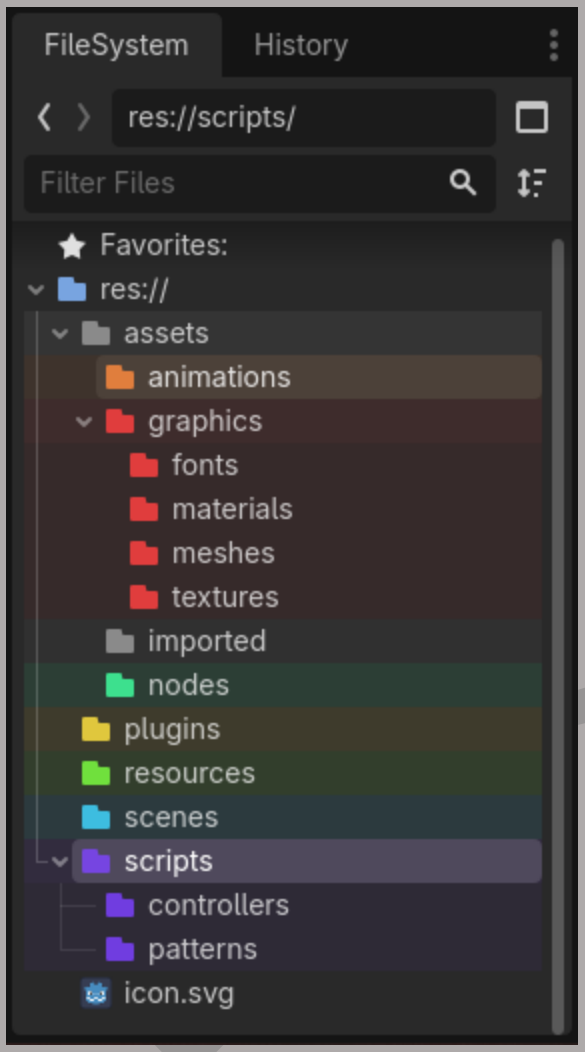
19 Workflow

This chapter will contain some sections of how to effectively, comfortably and productively work in Godot Engine, like: some tips of how to sort FileSystem, in order not to be confused during browsing through own assets in project, etc.

19.1 FileSystem Organization

There are some useful tips to better organize your own project in projects' filesystem:

- Create folders for each asset type, for example: **scripts**, doesn't matter which CSharp or GDScript – they should be located in scripts folder
- Better to set folder color, if you want



FileSystem Organization Example

19.X Blender Animations to Godot for FBX Export

Follow these steps to export Blender animations to Godot without any troubles:

1. Import animations
2. Rename bones
3. Set animation workspace
4. Drag two spaces: action editor and non-linear animation
5. Set anim_name and slot
6. Action -> Push down action
7. Shift+A -> All transforms to Deltas for right position, rotation and scale in Godot
8. Export -> Animation -> All actions (uncheck)
   -> NLA Strips (check)

Renaming Mixamo Bones

Additional Info

As an idea: if you need to make some procedural animation, but you need some keyframes as an animation starting pose, for example: you can set any kind of idle standing animation with weapon to 1 keyframe, and delete all other keyframes. Then export this animation to Godot.

REFERENCES

1. https://docs.godotengine.org/en/stable - Godot Engine documentation in English
2. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/ - Official Microsoft C#/.NET documentation
3. https://metanit.com/sharp - C#/.NET beginner/intermediate/advances tutorials in Russian